

Software Engineering Project

DispatchNet:

Alessio Zanon

Leonardo Tonetta

Luca Fiorini

User Flow & Implementation Analysis

Contents

Purpose	2
1 TODO CLEAN CLASS DIAGRAM	3
2 TODO D3	4
2.1 Project Structure	4
2.2 Architecture	5
2.3 Implementation Language	5
2.4 Database	5
2.5 Dependencies	5
2.6 User Flows	5
2.6.1 User Management	5
2.6.2 Ticketing	7
2.6.3 Passenger	8
2.6.4 Train Company	9
2.6.5 Network Manager	10
2.7 Code Preview	11
2.8 Team roles	11

Readability notes

Bold words are system-important constructs.

Underlined nouns are actors of the system.

Italic words are special keywords to denote important limits of the project.

[Hyperlinks](#) in reference to sections and subsections will be blue.

[Hyperlinks](#) to external web content will be in magenta.

Purpose

This document describes all the relevant information regarding the development of Project "TRE", which implements a rail transport management system.

The purpose of this document is to:

- Present User Flows related to the actions of performed by all users.
- Describe the (Placeholder) API interface with the payment processor.
- Accompany the documentation of the implemented processes.
- Explain and justify important implementations decisions taken.
- TODO other?

Chapter 1

TODO CLEAN CLASS DIAGRAM

Original, up-to-date files of D1 and D2 available at our [Github here](#) and [Github here](#) respectively

Chapter 2

TODO D3

2.1 Project Structure

The structure of the project is shown in **TODO: figure.x**. It is composed of:

- the main modules:
 - departure: **TODO: ?**;
 - infrastructure: the network and its management;
 - path_finding: the algorithm to find a path between stations;
 - service_management: the management of services;
 - user_management: the management of users and their authentication.
- secondary tools:
 - csv_database: the actual database and its management;
 - IO_operations: input/output operation to interact with the user from terminal;
 - mail_service: integration with a possible mail service, as well as a temporary mock one;

TODO: add photo in folder structure

2.2 Architecture

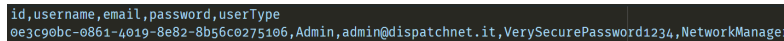
2.3 Implementation Language

2.4 Database

To save data between different we used a database saved as several CSV files. It will be loaded at startup, containing at least the account of the network manager, with default information. The data will be managed within the code and saved to the database only at the end of the program.

The database contains one file for each set of data:

- authenticated users
- junctions
- lines
- station_data
- **TODO: add if necessary**



```
id,username,email,password,userType
0e3c90bc-0861-4019-8e82-8b56c0275106,Admin,admin@dispatchnet.it,VerySecurePassword1234,NetworkManager
```

Original image can be found on our [Github here](#)

TODO: add database images

2.5 Dependencies

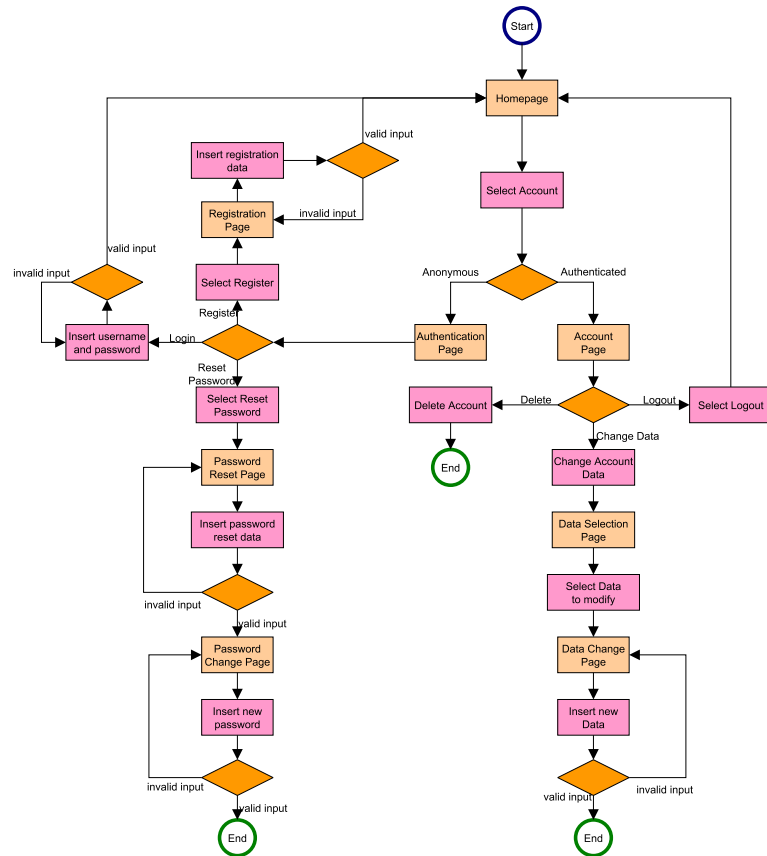
2.6 User Flows

2.6.1 User Management

The following user diagram describes the actions related to the user management and user authentication, common to all the users.

It provides different pages depending on the type of user. An authenticated user (I.E. passenger or train company, not network manager) can delete, view data (Account Page), change data or logout (also the network manager).

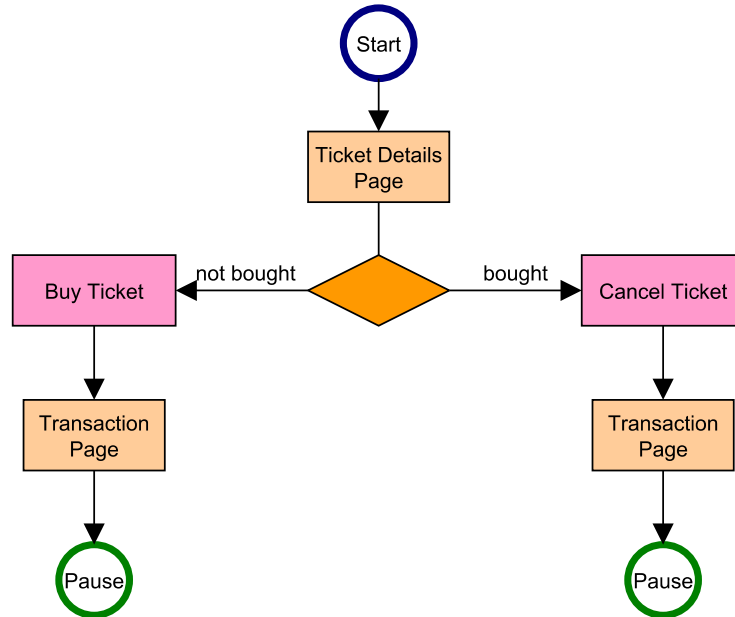
An anonymous user can register, reset its password or login.



Original image can be found on our [Github here](#)

2.6.2 Ticketing

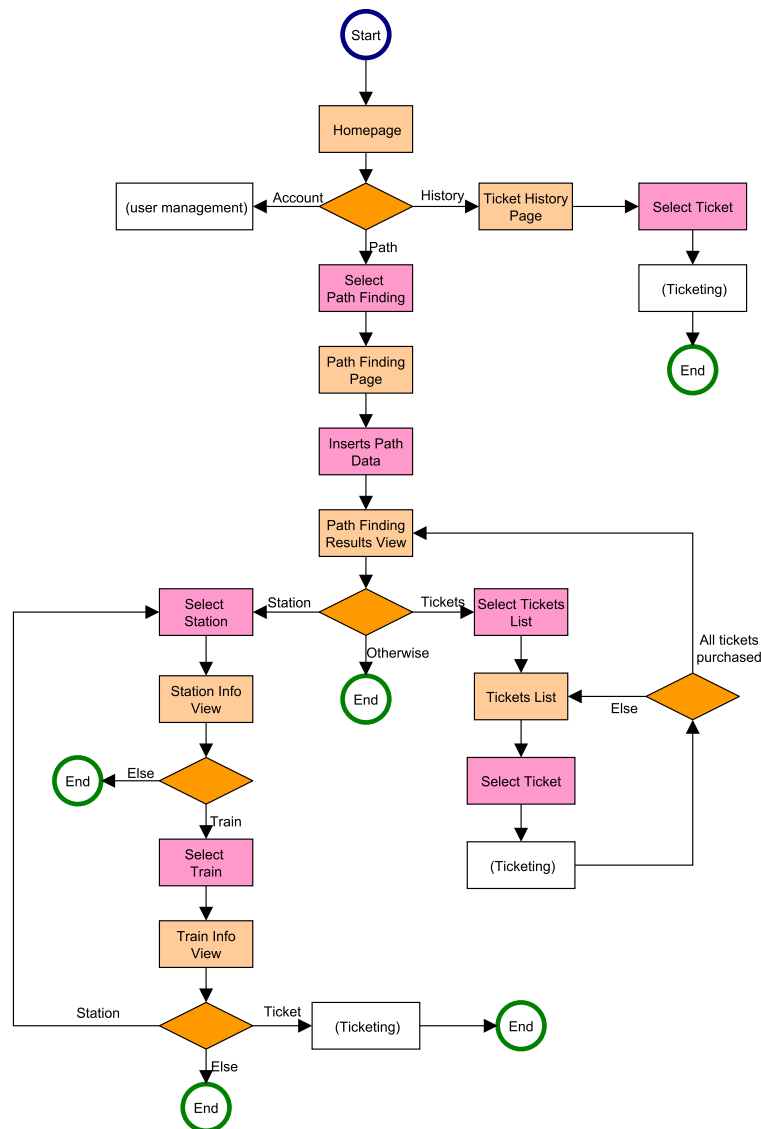
The following user diagram describes the flow of the ticketing component, started from the [Passenger's](#) diagram.



Original image can be found on our [Github here](#)

2.6.3 Passenger

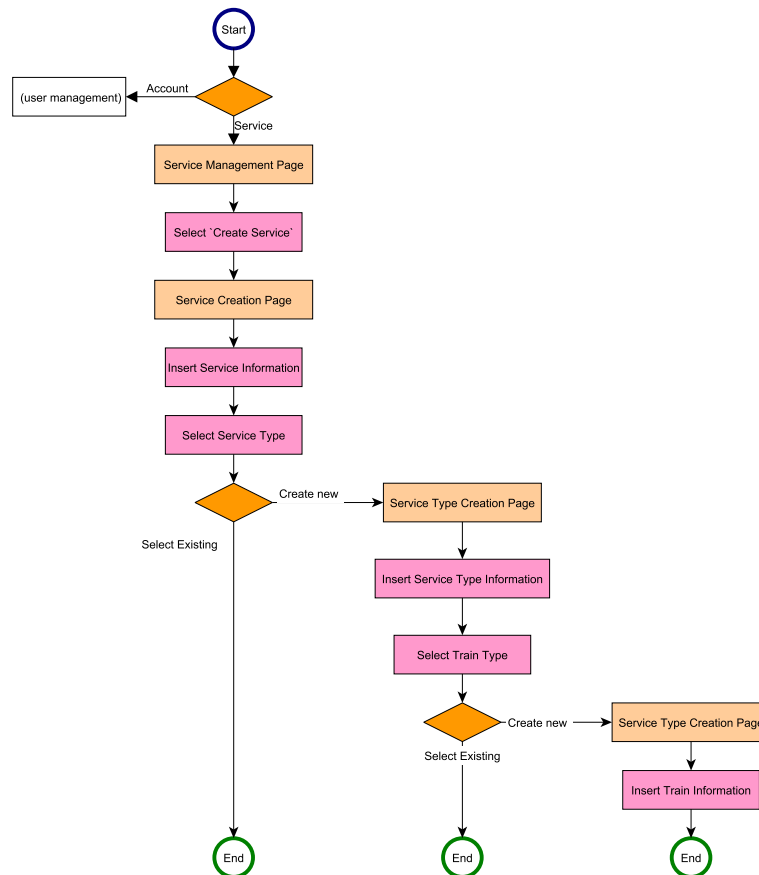
The following user diagram describes the actions the passenger can take. It can go to [User Management](#) **TODO: add hyperlink (everywhere)**, view its ticket history or start a path finding procedure. In the last two it can move to the [Ticketing](#) flow.



Original image can be found on our [Github here](#)

2.6.4 Train Company

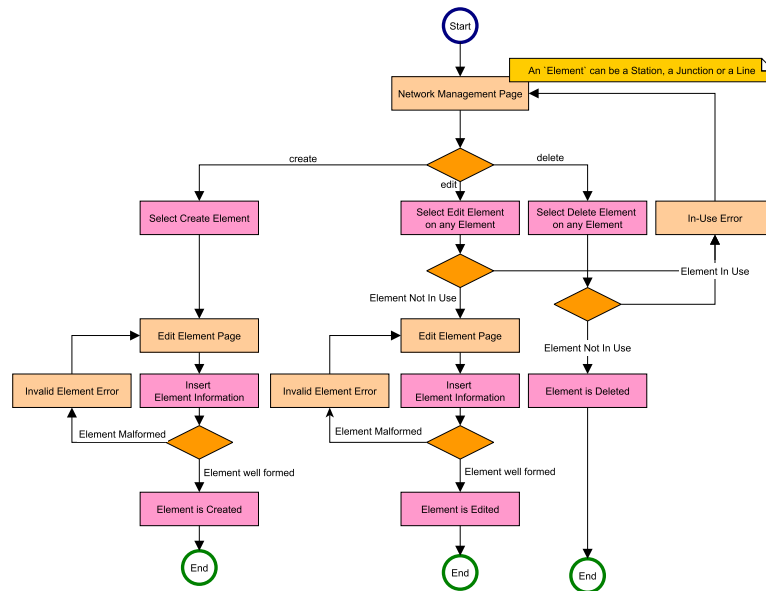
The following user diagram describes the actions the train company can take. It can go to [User Management](#) or to the service management.



Original image can be found on our [Github here](#)

2.6.5 Network Manager

The following user diagram describes the actions the network manager can take: managing the network and its elements.



Original image can be found on our [Github here](#)

2.7 Code Preview

2.8 Team roles